

AI Services B2_WA4_Medium Q2

Test Summary

- No. of Sections: 1
- No. of Questions: 1
- Total Duration: 900 min

Section 1 - COD

Section Summary

- No. of Questions: 1
- Duration: 900 min

Additional Instructions: None

Q1. EMPLOYEE POLICY QUESTION ANSWERING USING SECTION-ATTRIBUTED RAG

PROBLEM STATEMENT

Halcyon Logistics Pvt Ltd publishes its human resources rules as a single policy handbook. Employees routinely need answers to narrow questions on leave, expenses and conduct, and searching the handbook end to end is slow. An unattributed answer is of little practical use, because a decision on leave, reimbursement or disciplinary action cannot be claimed or contested unless the employee knows which policy section produced it.

The company requires a terminal application that answers an employee's question from the handbook and reports the sections that support the answer.

OBJECTIVE

Build a Retrieval-Augmented Generation pipeline that ingests the handbook PDF, divides it into overlapping chunks that retain their parent section, indexes those chunks in a ChromaDB vector store with section metadata, retrieves the excerpts most relevant to an employee's question, and generates a grounded answer accompanied by the cited section titles.

FILE STRUCTURE

```
Project
|-- data
|   |-- employee_policy_handbook.pdf
|-- .env
|-- main.py
|-- requirements.txt
`-- tests.py
```

PURPOSE OF FILES

main.py — Implements the retrieval-augmented generation pipeline and the terminal execution block.

tests.py — Contains the automated test cases that validate the implementation.

requirements.txt — Lists the Python package dependencies required by the project.

.env — Stores the Gemini API key and the model name as environment variables.

data/employee_policy_handbook.pdf — Employee policy handbook that serves as the knowledge base.

DOCUMENT DESCRIPTION

The knowledge base is a three page PDF titled Employee Policy Handbook 2026-27 of Halcyon Logistics Pvt Ltd. It contains ten numbered sections covering employment, working hours, leave, remote work, appraisal, payroll, travel, training, information security and grievance handling.

Every section heading occupies its own line and begins with a sequential number followed by the section title. Each heading is followed by a single paragraph of continuous prose stating the policy. The first page opens with the

handbook title and company name, which appear before the first heading. The document contains no tables, images or multi column layout, and every policy carries specific figures such as days, percentages, amounts and limits.

DOCUMENT SCHEMA

Field	Type	Description
Section Number	Integer	Sequential number from 1 to 10 that opens every heading line
Section Title	Text	Name of the policy area, on the same line as the section number
Section Body	Text	A single paragraph of 600 to 800 characters stating the policy

DOCUMENT CONTENT

Section Number	Section Title	Content Summary
1	Employment and Probation	Probation period, confirmation, notice period and joining formalities
2	Working Hours and Attendance	Weekly schedule, core hours, late arrival rule and overtime
3	Leave Entitlement	Leave quotas, carry forward limit, medical certificate and encashment
4	Remote Work and Hybrid Policy	Remote days per week, eligibility, internet allowance and withdrawal
5	Performance Appraisal and Increments	Review cycle, rating scale, increment range and review request
6	Compensation and Payroll	Salary credit date, pay structure, provident fund and variable pay
7	Travel and Expense Reimbursement	Approval lead time, hotel and meal limits, mileage and claim deadlines
8	Training and Certification Support	Learning budget, annual learning hours, service agreement and exam fees
9	Information Security and Device Use	Password rules, device loss reporting, awareness course and device return
10	Grievance and Disciplinary Procedure	Escalation path, disciplinary stages, show cause notice and harassment inquiry

TASK

FUNCTION: LOAD_AND_CHUNK_DOCUMENT

FUNCTION SIGNATURE

```
load_and_chunk_document(document_path)
```

PARAMETERS

- **document_path** — String. The relative path to the handbook PDF inside the data directory, supplied by the caller.

PURPOSE

Converts the handbook PDF into overlapping text chunks, each of which retains the title of the section it belongs to.

REQUIREMENTS

- Extract the text of every page and treat it as one continuous body.
- Divide the body at its numbered section headings so that a heading and the paragraph beneath it form a single unit.
- Exclude any content appearing before the first heading, as it belongs to no section.
- Collapse the line breaks inside each section body into single spaces before windowing.
- Reduce each section body to windows of 500 characters that advance in steps of 420 characters, so that consecutive windows of the same section share 80 characters.

- Confine every window to a single section.
- Discard any chunk shorter than 60 characters after stripping.

RETURNS

A list of dictionaries. Each dictionary holds the heading of the section the chunk belongs to and the chunk text, under the keys **section** and **text**. The heading is kept exactly as printed, including its number (for example, 3. Leave Entitlement).

FUNCTION: RETRIEVE_SECTIONS

FUNCTION SIGNATURE

```
retrieve_sections(question, embedding_model, collection)
```

PARAMETERS

- **question** — String. The natural language question typed by the employee, received from the orchestrator.
- **embedding_model** — SentenceTransformer instance. The pre-loaded embedding model created in the orchestrator, used to convert the question into a vector of the same dimensionality as the stored chunk embeddings.
- **collection** — ChromaDB collection object. The in-memory collection created in the orchestrator, already populated with every chunk embedding, chunk text and section label.

PURPOSE

Finds the handbook excerpts most semantically relevant to the question and returns them with their section labels and relevance scores.

REQUIREMENTS

- Embed the question with the model supplied and query the collection for the four closest chunks.
- Read the returned documents, metadata and distances together so that every chunk keeps its own section label.
- Convert each cosine distance into a similarity score (1 minus the distance) rounded to two decimal places, where a higher value denotes a closer match.
- Return the results ordered most relevant first.

RETURNS

A list of four dictionaries ordered by descending similarity, each holding the excerpt text, its section title and its similarity score, under the keys **chunk**, **section** and **similarity**.

FUNCTION: AUGMENT_PROMPT

FUNCTION SIGNATURE

```
augment_prompt(question, retrieved_chunks)
```

PARAMETERS

- **question** — String. The employee's original question, passed through from the orchestrator.
- **retrieved_chunks** — List of dictionaries. The output of `retrieve_sections`, each entry carrying an excerpt, its section title and its similarity score.

PURPOSE

Assembles the grounded prompt that will be submitted to the language model.

REQUIREMENTS

- Present the excerpts as a numbered context block with each entry visibly labelled with the section it came from.
- Exclude the similarity score from the prompt.
- Instruct the model to answer only from the supplied excerpts and to name the supporting section.
- Instruct the model to state that the matter is not covered when the excerpts do not contain the answer.
- Include the employee's question in the prompt.

RETURNS

A string containing the complete prompt.

FUNCTION: GENERATE_ANSWER

FUNCTION SIGNATURE

```
generate_answer(prompt, client)
```

PARAMETERS

- **prompt** — String. The assembled prompt produced by `augment_prompt`.
- **client** — Gemini client object. An authenticated client created in the orchestrator from the API key held in the environment.

PURPOSE

Submits the prompt to the language model and returns the generated answer.

REQUIREMENTS

- Send the prompt through the client supplied.
- Read the model name from the environment rather than hard coding it.
- Return the response text free of leading and trailing whitespace.

RETURNS

A string containing the answer generated by the model.

FUNCTION: POLICY_QA_PIPELINE

FUNCTION SIGNATURE

```
policy_qa_pipeline(question, document_path)
```

PARAMETERS

- **question** — String. The employee's question, entered at the terminal or supplied directly by a caller.
- **document_path** — String. The relative path to the handbook PDF inside the data directory.

PURPOSE

Coordinates the complete pipeline and assembles the structured result, including the list of cited sections.

REQUIREMENTS

- Chunk the handbook and embed every chunk in a single batch operation using the `all-MiniLM-L6-v2` SentenceTransformer model.
- Create an ephemeral in-memory ChromaDB collection with cosine distance, removing any collection surviving from an earlier call before creating it.
- Supply the embeddings explicitly rather than relying on the collection's own embedding function.
- Store each chunk with its text as the document and its section title as metadata, keeping identifiers, embeddings, documents and metadata aligned.
- Run the three pipeline stages in order: retrieval, prompt augmentation and answer generation.
- Build the citation list from the section titles of the retrieved chunks with duplicates removed and retrieval order preserved.

RETURNS

A dictionary holding the original question, the retrieved chunks, the cited section titles and the generated answer, under the keys **question**, **retrieved_chunks**, **sources** and **answer**.

MAIN BLOCK

REQUIREMENTS

- Read the question from the terminal and discard the surrounding whitespace.
- Reject an empty entry with a message and perform no further processing.
- Run the pipeline against the handbook.
- Print the question, the retrieved sections each with its similarity score, the comma separated list of cited sections, and the generated answer.

OUTPUT

GENERATED FILES

No files are generated by this program. All results are printed to the console.

CONSOLE OUTPUT

The program prints the question as entered, the four retrieved sections each with its similarity score, the comma separated list of cited sections, and the generated answer. When the entry is empty, only the message **Question cannot be empty.** is printed.

EXPECTED OUTPUT

```
Enter your question: How many days of earned leave can be carried forward to the next year?
Question: How many days of earned leave can be carried forward to the next year?
Retrieved Sections:
1. 3. Leave Entitlement (similarity: 0.62)
2. 3. Leave Entitlement (similarity: 0.44)
3. 8. Training and Certification Support (similarity: 0.31)
4. 1. Employment and Probation (similarity: 0.27)
Sources: 3. Leave Entitlement, 8. Training and Certification Support, 1. Employment and Probation
Answer:
Up to 12 days of earned leave may be carried forward to the next year. Any balance
above this limit lapses on 31 December. Casual leave cannot be carried forward. This
is stated in section 3. Leave Entitlement.
```

The similarity values and the wording of the generated answer vary between runs.

IMPLEMENTATION STEPS

Environment Setup

- 1.1 Navigate to project directory:
`cd Project`
- 1.2 Install dependencies from requirements.txt:
`pip install -r requirements.txt`
- 1.3 Implementing Solution:
`python3 main.py`
- 1.4 To Run Testcases
`python3 -m pytest tests.py -vv`